SOLVEBENCH

Solving $A\vec{x} = \vec{b}$ at Scale

A Diagonal-Selection Pipeline for Stationary Iterative Solvers

Every blackout-prevention study and every bridge safety check reduces to this exact equation. We benchmarked 16 solvers on 927 real matrices, found what breaks, and built a fix for it.

Section A2 · Roll Numbers 2105037 · 2105038 · 2105042 · 2105047 · 2105054

Where This Talk Is Going

This is the full story, in the order it actually happened. Seven parts.

1. **The problem** we are solving, and the paper we extend
2. **What we implemented** from that paper
3. **Scaling the experiment** from 5 unknowns to 927 real matrices
4. **Baseline results** across all 16 solvers
5. **The story of our novelty**, milestone by milestone, starting from our first idea (APK)
6. **Honest limits**: what we do not claim
7. **Where things stand**, and what is next

Every number in this deck is measured and traceable to a table in the repository. Where a result is still being measured, we say so.

PART 1

The Problem We Solve

Why $A\vec{x} = \vec{b}$ is the one equation every simulation eventually hits

What Problem Are We Solving?

Almost every engineering simulation, whether it is a power grid, a bridge, a circuit, or an airflow model, reduces to the same piece of algebra:

$$A\vec{x} = \vec{b}$$

A is huge and **sparse**: almost every entry is exactly zero, because most parts of a physical system only interact with their close neighbours.

Direct methods

Factor $A = LU$, then solve two easy triangular systems. Exact answer, fixed number of steps. Can need huge memory for fill-in.

Iterative methods

Start from a guess, improve it step by step, stop once the residual is small. Cheap per step. May never get closer at all.

The whole project, in one sentence

Whether an iterative method converges depends entirely on the structure of A. That dependency is what this project measures, on real data.

Base Paper: What It Studies

“Comparative Analysis of Jacobi and Gauss-Seidel Iterative Methods”

P. Khrapov and N. Volkov

International Journal of Open Information Technologies, vol. 12, no. 2, pp. 23–34, 2024

- Works out, by hand, exactly which **2 and 3 unknown** matrices make Jacobi and Gauss-Seidel converge.
- Checks that pattern statistically on **random matrices**, up to 5 unknowns, generated by computer.
- Shows the two methods do **not** always share a convergence region: which one wins depends on the matrix.

The gap this leaves open

The paper never once touches a matrix from a real engineering problem. Everything is either worked out by hand at toy size, or generated at random. Random numbers do not look like real data: they carry none of the structure that comes from how a real circuit or bridge is actually built.

The Gap, and Our Question

Nobody has checked whether the paper's conclusions survive contact with matrices that engineers actually have to solve, at the sizes those methods are actually used at.

Our question: do Jacobi and Gauss-Seidel behave the way the theory says on real engineering matrices, and where do they stand once you put the solvers an engineer would actually reach for beside them?

This is an **exploratory and comparative study**: we benchmark direct, stationary, Krylov, and preconditioned solvers against 927 real matrices, and we build one genuinely new preprocessing idea on top of what we find.

PART 2

What We Built From the Paper

Jacobi, Gauss-Seidel, SOR, and the convergence theory behind them

Jacobi and Gauss-Seidel: the Update Rules

Split $A = D + L + U$: the diagonal, everything below it, everything above it.

Jacobi

$$\vec{x}^{(k+1)} = D^{-1}\left(\vec{b} - (L+U)\vec{x}^{(k)}\right)$$

Every entry uses only values from the **previous** full pass. One matrix-vector product per step.

Gauss-Seidel

$$\vec{x}^{(k+1)} = (D+L)^{-1}\left(\vec{b} - U\vec{x}^{(k)}\right)$$

Each row can use **already-updated** values from earlier rows in the same pass. Usually faster than Jacobi.

Both need one thing to even exist

Both divide by the diagonal entries of D. If any $a_{ii} = 0$, that division is impossible: the method is not slow, it is **undefined**.

SOR, and Whether Any of This Converges

SOR (Successive Over-Relaxation) adds one tuning number, ω, that lets the update overshoot on purpose:

$$(D + \omega L)\,\vec{\delta} = \omega\,\vec{r}, \qquad \vec{x} \leftarrow \vec{x} + \vec{\delta}$$

$\omega = 1$ reduces exactly to Gauss-Seidel. The base paper's proposal and our own results (Part 5) both use this third method.
Convergence itself is governed by one number, the spectral radius $\rho(T)$ of the method's iteration matrix T:

$$\rho(T) < 1 \;\Rightarrow\; \text{converges, eventually} \qquad \rho(T) \geq 1 \;\Rightarrow\; \text{does not converge}$$

$\rho(T)$ also sets the speed: reaching tolerance ε takes roughly $\log(\varepsilon)/\log(\rho(T))$ steps. $\rho = 0.5$ needs about 27 steps; $\rho = 0.999$ needs about 18,000.

The Full Method Roster: 16 Solvers, 5 Families

We did not stop at the two methods the paper studies. Without a floor and a ceiling, “Jacobi is slow” is not a meaningful sentence.

Family	Methods	Count
Direct, hand-written	Gauss elimination, Gauss-Jordan, LU, Cholesky	4
Direct, library	`spsolve`, `splu` (SuperLU)	2
Stationary	**Jacobi, Gauss-Seidel, SOR**	**3**
Krylov	Conjugate Gradient, BiCGSTAB, GMRES(30)	3
Preconditioned	ILU only, ILU-BiCGSTAB, ILU-GMRES(30), ILU-Krylov (dispatched)	4

The last row, ILU-Krylov (dispatched), is our original course proposal, APK. Its story is told honestly in Part 5.

PART 3

Scaling the Experiment Up

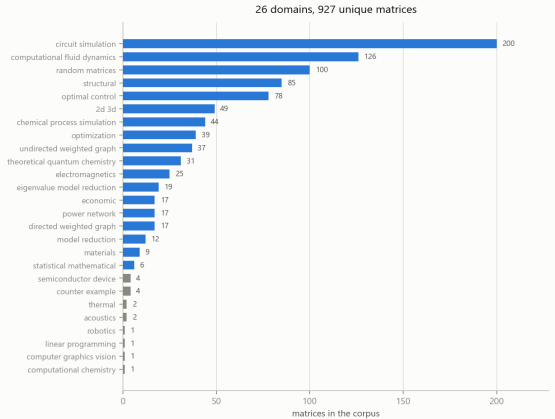
From 5 unknowns and random numbers to 927 real matrices at $n \leq 10{,}000$

How Much We Scaled It Up

	The paper	This project
Matrix source	hand-derived, then random	real engineering data
Matrix count	a handful, then a random batch	**927 unique matrices**
Matrix size	up to 5 unknowns	**up to 10,000 unknowns**
Domains	none (random numbers)	**26 real domains**
Methods compared	Jacobi, Gauss-Seidel	**16 methods, 5 families**

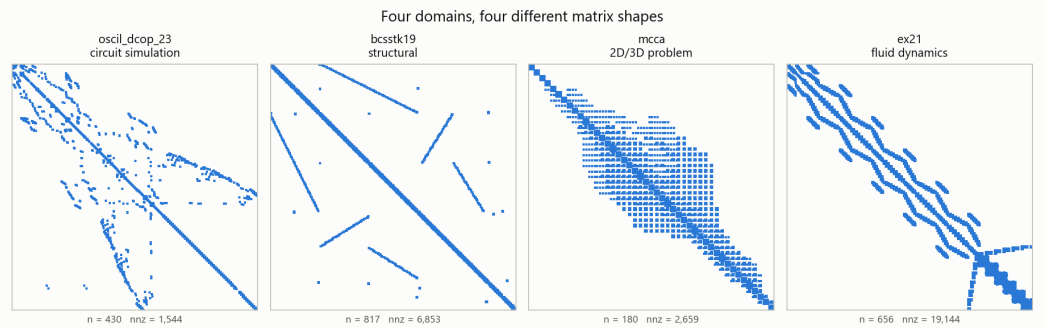
Source: the SuiteSparse Matrix Collection, the standard public library of real sparse matrices contributed by engineers from actual projects.

26 Domains, 927 Matrices



One domain has 200 matrices, eight have fewer than 5. Every per-domain claim in this project reports its own *n* for exactly this reason.

What the Matrices Actually Look Like



A spy plot: one dot per nonzero entry, position only. A circuit, a structural mesh, a 2D/3D problem and a fluid-dynamics discretisation do not just differ in numbers, they differ in shape.

Corpus Bookkeeping: Being Honest About 930 Files

- 930 files downloaded. **3 are exact duplicates** of other files under a different name ⇒ **927 unique matrices**.
- **6 files were truncated downloads**: each file's own header declared more nonzero entries than the file contained, so they silently failed to load. Usable corpus: **924**.
- **100 of the 930 are random matrices we generated ourselves**, sizes 5 to 300, specifically to reproduce the base paper's own validation method (Part 4 shows what that comparison found).
- One domain was spelled two ways in the source data, splitting it in two by accident. We merge it back.

Why this matters

A downloaded file that is never checked against its own declared size can silently break a result months later. This is one of the mistakes we made and fixed (Part 5, and the closing section).

How We Decide a Solve Actually Succeeded

A solver's own claim of success is **recorded, never trusted**. Success is decided afterwards, by us, from the residual we measure independently:

$$\text{relative residual} = \|A\vec{x} - \vec{b}\| / \|\vec{b}\| \leq 10^{-8} \;\Rightarrow\; \text{solved}$$

- **solved**: residual verified below tolerance
- **inaccurate**: solver claimed success, residual disagrees
- did_not_converge: solver admitted failure
- diverged: the iterate blew up
- not_applicable: undefined on this matrix
- structurally_singular: no unique solution
- skipped_too_large: deliberately not attempted
- error: an unexpected failure, recorded

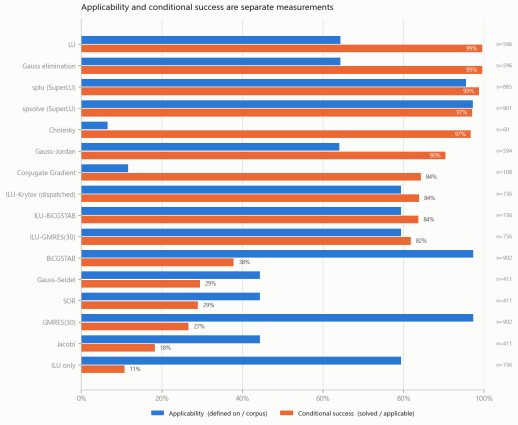
The very first version of this project trusted a solver's own success flag, and passed 57 solves whose actual residual reached 9.71×10^{12}.

PART 4

Baseline Results Across All 16 Solvers

What each method can actually do on 927 real matrices

Applicability vs. Conditional Success, All 16 Methods



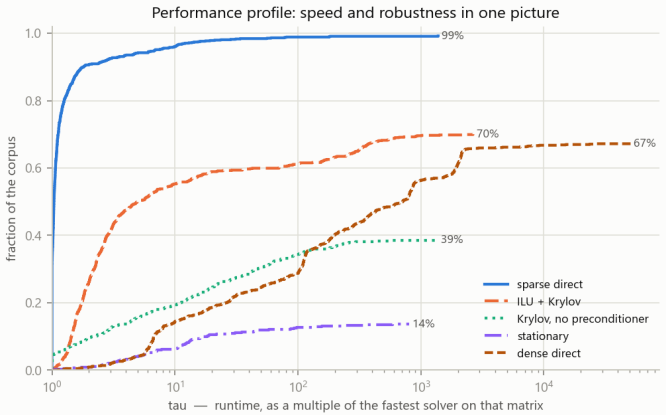
Two rates, never multiplied. Conjugate Gradient is applicable on only 11.7% of the corpus, but solves 84.3% of what it is allowed to try.

The Full Numbers, Ranked by Systems Solved

Method	Applic.	Solved	Applicability	Cond. success
`spsolve` (SuperLU)	901	**874**	97.2%	97.0%
`splu` (SuperLU)	885	874	95.5%	98.8%
ILU-Krylov (dispatched)	736	617	79.4%	83.8%
Gauss elimination / LU	596	593	64.3%	99.5%
BiCGSTAB	902	341	97.3%	37.8%
Gauss-Seidel	411	121	44.3%	29.4%
Conjugate Gradient	108	91	11.7%	84.3%
Jacobi	411	75	44.3%	18.2%

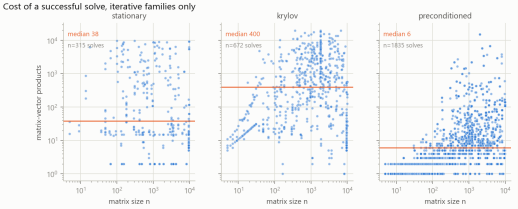
All 16 rows, with every denominator, are in `RESULTS.md`.

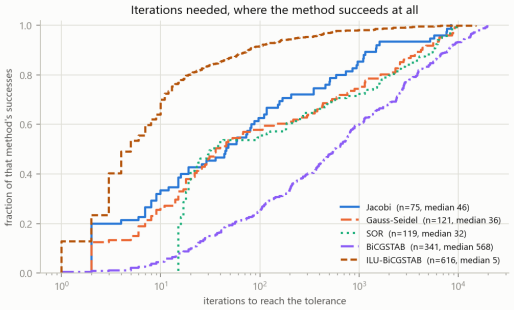
Speed and Robustness, in One Picture



A Dolan-Moré performance profile, the standard tool for comparing solvers. Height at τ=1: how often a method is outright fastest. The right-hand plateau: how much of the corpus it can solve at all.

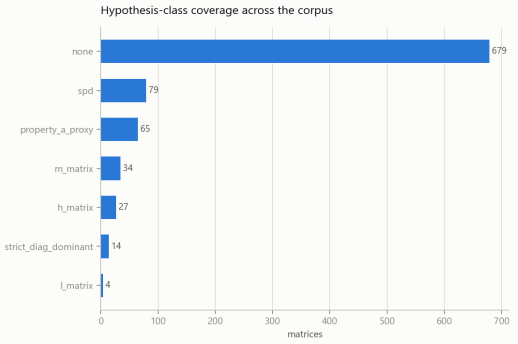
Cost and Iteration Count

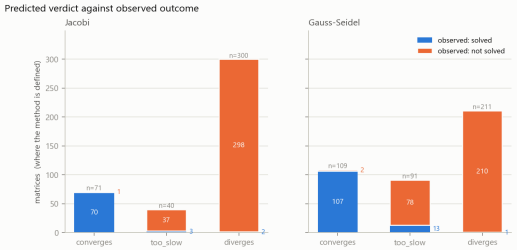




Cost is counted in matrix-vector products, not seconds: shared Kaggle hardware makes wall-clock time unreproducible. One preconditioned step costs far more arithmetic than one Jacobi step, so fewer iterations is not the same as less work.

Does the Textbook Theory Predict the Outcome?





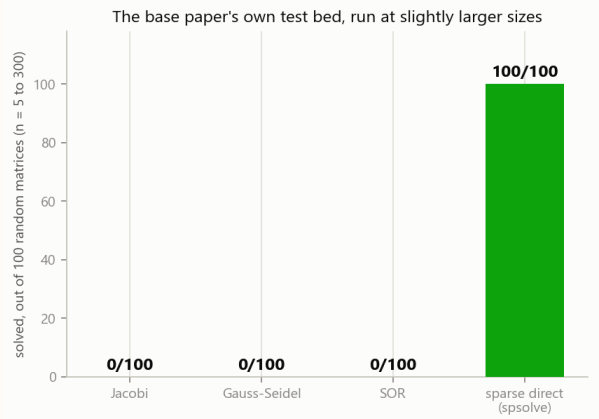
Stein-Rosenberg (1948) and Householder-John (1958) cover part of the corpus. “too_slow” is the case the textbook criterion cannot express: $\rho < 1$, so convergence is guaranteed, but not inside any usable iteration budget.

The Single Most Important Fact We Found

491 of 927

matrices leave Jacobi, Gauss-Seidel and SOR mathematically undefined, all at once, because of one zero on the diagonal

The Base Paper's Own Test Bed, at Real Scale



100 random matrices, sizes 5 to 300, exactly the kind the base paper validates its theory on. $\rho(T_{Jacobi})$ grows roughly in proportion to size on random matrices, so the paper's own method stops working almost as soon as it is scaled up even slightly.

PART 5

The Story of Our Novelty

Ten milestones, in the order they actually happened, starting from our first idea

Milestone 0: Our Starting Point, APK

Our original proposal: inspect the matrix, then dispatch to the right Krylov method.

$$\text{symmetric} \Rightarrow \text{PCG}, \qquad \text{otherwise} \Rightarrow \text{BiCGSTAB}$$

both preconditioned with ILU.

What we found when we measured it honestly

This exact rule already has a name: the dispatch logic in Barrett et al., **Templates** (SIAM, 1994), and the default behaviour of PETSc and MATLAB's backslash operator. It was not new.

ILU-Krylov, dispatched: **617** of 736 solved. Always ILU-BiCGSTAB, no dispatch at all: **616**.

One matrix, out of 736.

Kept today as an honestly labelled baseline, not as our novelty. This is why the project needed a real contribution.

Milestone 1–2: an Honest Harness, and the 491

Before any new idea could be tested fairly, the benchmark itself had to be trustworthy. We rebuilt it, fixing five separate problems (Part 7 lists all eight mistakes found across the whole project). Once it could be trusted, the very first full sweep produced the number that reshaped everything that followed:

491 of 927 matrices (53%) have at least one zero on the diagonal.
Jacobi, Gauss-Seidel and SOR cannot even attempt them.

This turned the question from “is Jacobi slow?” into a much bigger one: **can we make Jacobi applicable in the first place?**

Milestone 3: the Idea, Choose the Diagonal

Row order is arbitrary. It was decided by the meshing software or the circuit numbering convention, never by the solver that will run later. Permuting rows changes nothing about the answer:

$$A' = PA, \quad \vec{b}' = P\vec{b} \;\Rightarrow\; A'\vec{x} = PA\vec{x} = P\vec{b} = \vec{b}'$$

$A = \begin{pmatrix} 0 & 5 \\ 3 & 1 \end{pmatrix}$: a_{11}=0, undefined for every stationary method.

Swap the rows: $A' = \begin{pmatrix} 3 & 1 \\ 0 & 5 \end{pmatrix}$, and every method can now run. Same equations, same answer.

This single idea is the seed of everything in the rest of this part.

Milestone 4: Proving What Can and Cannot Work

Before searching for a good row order, we checked what kinds of change could possibly help, so the search would not waste time on something pointless.

- **Row scaling** leaves Jacobi's iteration matrix algebraically identical: $T'_J = I - (RD)^{-1}RA = T_J$, exactly.
- **Column scaling** is a similarity transform: never changes the spectral radius.
- **Symmetric permutation** barely moved it on real matrices: the fifth decimal place, $0.99012 \rightarrow 0.99011$.
- **Row permutation alone** is the only operation that changes which entries sit on D.

This proof, not a guess, is why we searched over row permutations specifically.

Milestone 5: the Objective, and a False Start

Choosing the diagonal is an **assignment problem**, solvable exactly.

MC64 (established)

maximise $\prod |a_{ii}|$

Built for pivot stability in **direct** solvers. Never looks at the rest of the row.

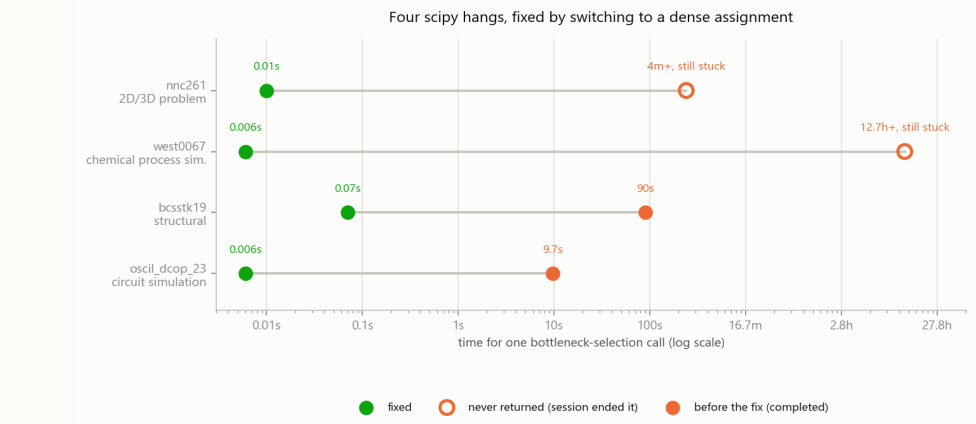
Bottleneck (ours)

minimise $\max_i \dfrac{\sum_{j \neq i} |a_{ij}|}{|a_{ii}|}$

Below 1 for every row **is** strict diagonal dominance, which guarantees convergence.

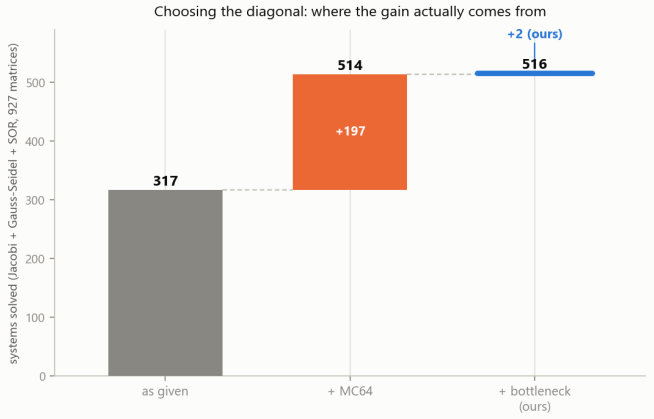
The false start: a second objective, min-sum, was meant as a natural contrast. Algebra shows $\sum \log(1+\text{ratio}) = \text{const} - \sum \log |a_{ii}|$: min-sum **is** MC64 under another name. Verified on 193 matrices, 193 matches.

Milestone 6: Four scipy Hangs



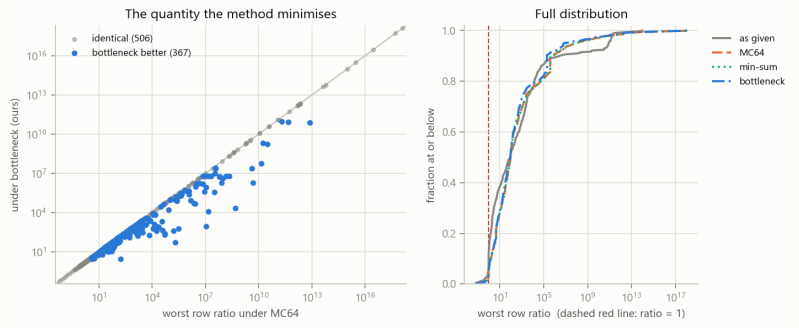
Two ready-made scipy matching routines hung, four separate times, in four separate ways, on real matrices. One of them stalled an entire 12.7-hour Kaggle session. The fix each time: a dense assignment instead.

Milestone 7: What Reordering Actually Buys



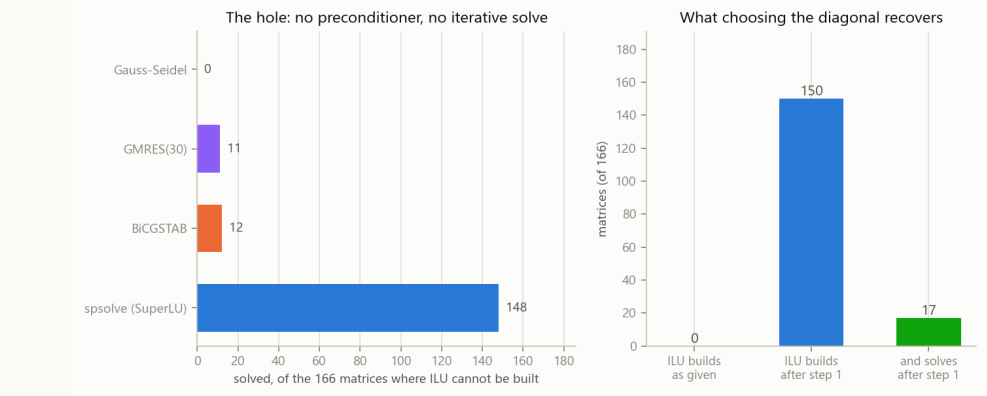
199 systems recovered, zero lost. But 195 of the 199 are recovered by **both** objectives: the idea of reordering is the large effect, our own choice of objective is the small one.

Milestone 7 (continued): Our Objective, Head to Head



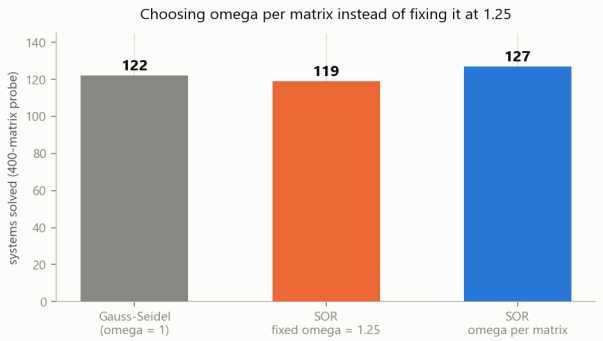
On the quantity bottleneck is built to minimise, it beats MC64 **367 to 0**, 506 ties, never once losing. What that win does not buy: not one further matrix in the corpus can be pushed under the diagonal-dominance threshold by any row permutation. 43 already were.

Milestone 8: the ILU Hole



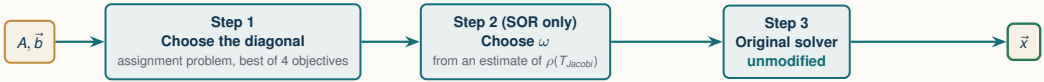
166 matrices where no ILU preconditioner could be built at all, taking the whole preconditioned family down together. Reordering makes 150 of the 166 buildable, and 17 solve outright.

Milestone 9: the Relaxation Factor, Chosen per Matrix



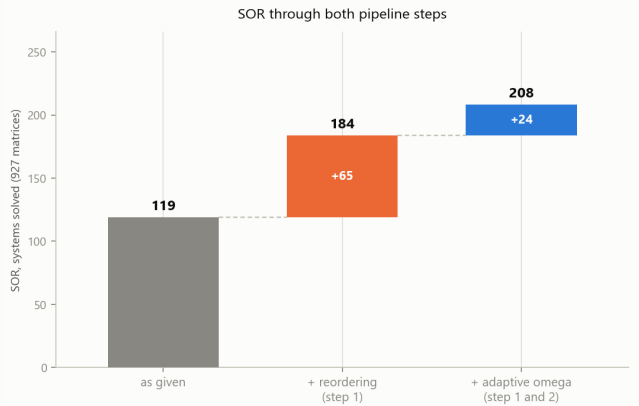
A fixed $\omega = 1.25$ is the wrong call in both directions at once. Estimating $\rho(T_{Jacobi})$ per matrix and applying Young's 1950 formula recovers 8 of the 20 systems where ω decides the outcome at all.

Milestone 10: Assembling the Pipeline



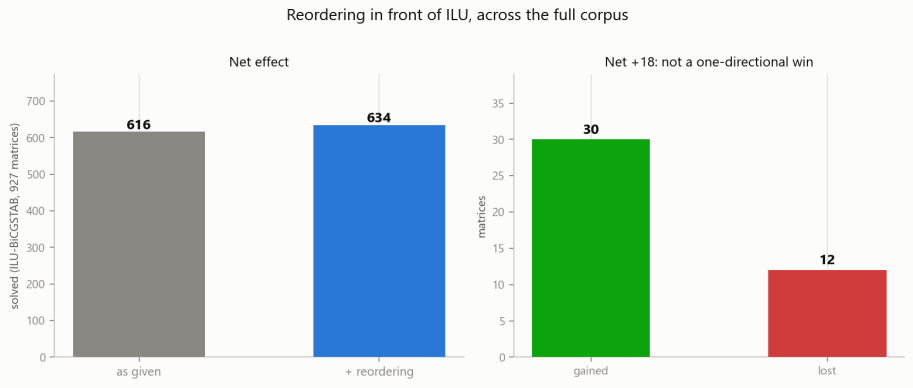
- Every arm is measured **separately** as well as **together**. “The pipeline helps” is not a result; **which step** helps, and by how much, is.
- Steps 1 and 2 never touch the solver itself. $A'\vec{x} = \vec{b}'$ solves the original system directly: nothing needs to be undone.
- **All seven arms have now run**, across all 927 matrices, 413.6 minutes, no hang. What that run found is on the next two slides.

Milestone 10 (continued): SOR Through Both Steps



Step 2 adds 24 on top of step 1, a 75% increase over the as-given baseline. Not loss-free: relative to step 1 alone it gains 36 and loses 12, because choosing ω per matrix has no “do nothing” fallback the way step 1 does.

Milestone 10 (continued): Reordering in Front of ILU, Whole Corpus



The 166-matrix hole (Milestone 8) could only show gains, since its baseline was zero. Across all 927, ILU-BiCGSTAB and ILU-Krylov both move on identical sets: 30 gained, 12 lost, net +18. Changing the diagonal changes what ILU factors against.

PART 6

Honest Limits

What we do and do not claim, stated plainly

What We Do Not Claim

- **We do not claim to have beaten MC64.** Against MC64 alone our portfolio is worth **+2** systems solved. The reordering **idea** carries 199; our own objective is a small part of it.
- **We do not claim min-sum is a second, independent idea.** It is MC64, proved algebraically and verified on 193 matrices.
- **We do not claim the diagonal-dominance guarantee is reachable here.** 43 matrices already have it; zero more can be reached by any row permutation on this corpus.
- **We do not claim a new iterative method.** This is a preprocessing pipeline in front of solvers that are otherwise completely unmodified.
- **We do not claim the full pipeline is loss-free.** Step 2 gains 36 and loses 12 on top of step 1 for SOR; reordering in front of ILU gains 30 and loses 12 across the whole corpus. Only step 1 alone is guaranteed never to lose.

What we do claim: a diagnosed cause, a principled objective with a proof behind it, and a measured, honest gain. That is what the story in Part 5 is built on.

PART 7

Where Things Stand

Everything planned has run. What comes next

Status Right Now

Piece	Status
16-method main sweep, 927 matrices	done
Spectral analysis (theory vs. reality)	done
Refinement study	done
Reordering study (stationary methods)	done
ILU-hole diagnosis (166 matrices)	done
Adaptive-omega probe (400 matrices)	done
Full 7-arm pipeline, whole corpus	done

Every planned run has finished. Nothing in this project is still running.

353 further SuiteSparse matrices have been downloaded into a separate folder for a possible future, larger run. They are deliberately not mixed into the 927 this project's numbers are measured on.

Eight Mistakes We Made, and Fixed

1. Trusted a solver's own success flag (57 false successes, residual 10^{12}).
2. Divided success rates by the whole corpus instead of by applicability.
3. Gave iterative refinement to one method only.
4. Gauss-Seidel and SOR re-factored inside the loop, inflating runtime 11–14x.
5. A divergence guard that killed genuine, slow-to-turn convergences.
6. Two copies of the library quietly drifted apart.
7. Downloaded files were never checked against their own declared size.
8. Trusted two scipy routines without testing them on real data first.

Full detail on every one of these, with the number each fix changed, is in `Presentation/explaination.md` and `docs/GUIDE.md`.

Thank You

Questions?

SolveBench · CSE 402, Section A2

2105037 · 2105038 · 2105042 · 2105047 · 2105054